Heart Disease Risk Prediction

An end-to-end MLOps pipeline: experiment tracking, CI/CD, containerisation, Kubernetes and monitoring

Course	Machine Learning Operations (MLOps), AIMLCZG523 - Assignment 1
Student	Your full name BITS ID: Your BITS ID
Code repository	https://github.com/pnand5/heart-disease-mlops
Deployed API	http://localhost:8080 (Docker Desktop Kubernetes, local; see section 9)

1. Overview

The goal is a reproducible, automated and observable machine-learning service that predicts whether a patient has heart disease from routine clinical measurements. I used the UCI Heart Disease (Cleveland) dataset: 303 patients, 13 predictors and a target that I binarised into *no disease* versus *disease*. The final model is a **Logistic Regression** inside a single scikit-learn pipeline, served by a FastAPI container on a local Kubernetes cluster, with request logging, Prometheus metrics and a Grafana dashboard.

Model	CV ROC-AUC (5-fold)	Test ROC-AUC	Test accuracy	Test recall
Logistic Regression (selected)	**0.902 +/- 0.014**	0.966	0.885	0.929
Random Forest	0.901 +/- 0.023	0.944	0.836	0.821
Gradient Boosting	0.875 +/- 0.030	0.951	0.885	0.929

Logistic Regression and Random Forest are statistically tied on cross-validation, and the test split holds only 61 patients, so the choice rests on simplicity and interpretability rather than on a meaningful accuracy gap (section 5).

2. Architecture

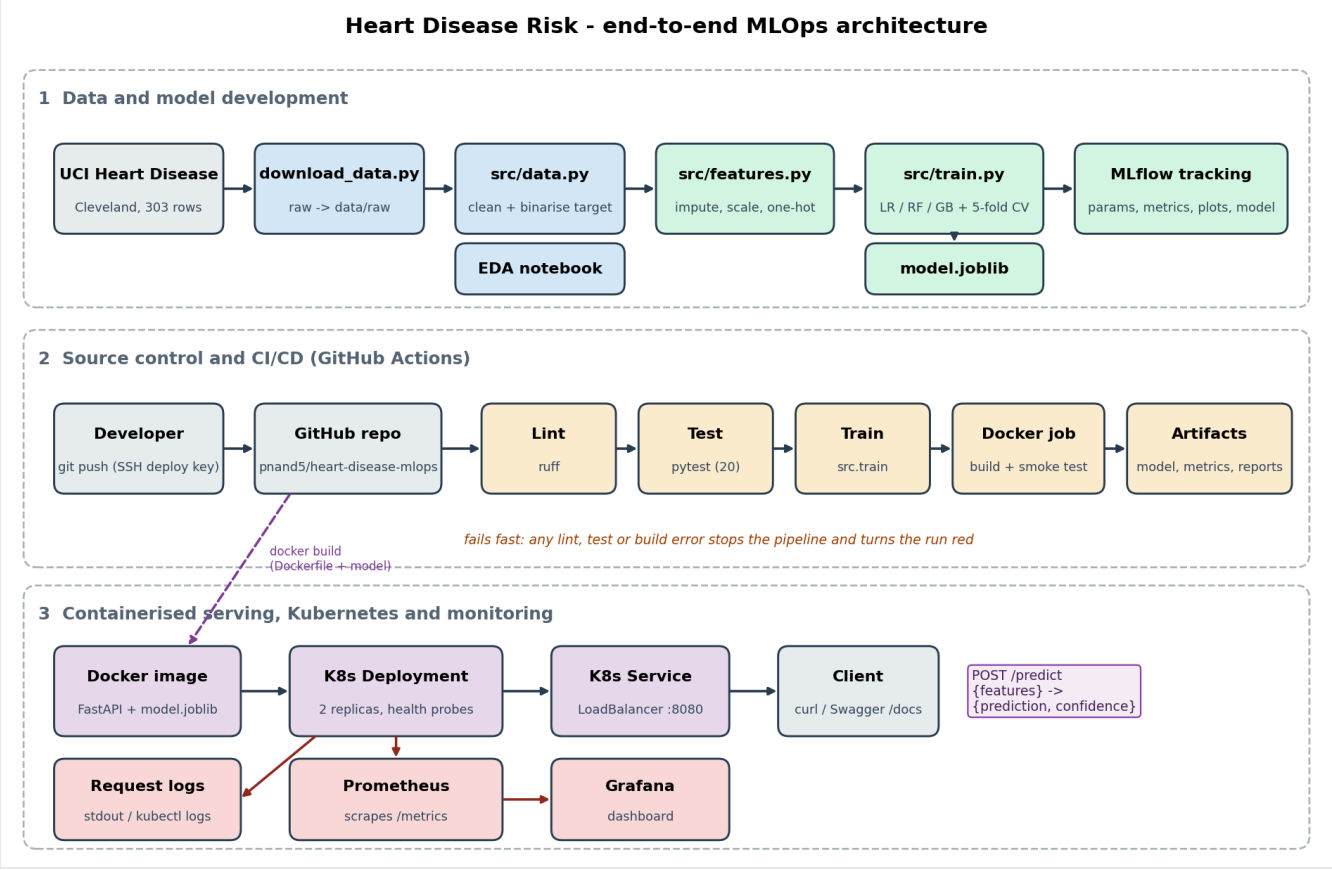


Figure 1 - End-to-end architecture: data and model development, CI/CD on GitHub Actions, and containerised serving with monitoring.

Code flows from the developer to GitHub, where Actions lint, test, train and smoke-test a container on every push. The same Dockerfile produces the image that Kubernetes runs; Prometheus scrapes its `/metrics` endpoint and Grafana visualises them.

3. Setup and installation

Requirements: Python 3.10; Docker Desktop (with Kubernetes enabled) for the container, cluster and monitoring steps. Everything below runs from a clean clone using only `requirements.txt`.

```
git clone https://github.com/pnand5/heart-disease-mlops.git && cd heart-disease-mlops
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

python scripts/download_data.py   # UCI file -> data/raw/
python -m src.data                # clean -> data/processed/heart_clean.csv
python -m src.train               # tune, evaluate, track (MLflow), save models/model.joblib
pytest -v && ruff check .         # 20 tests + lint
make mlflow-ui                    # tracking UI on http://localhost:5001
```

The Makefile wraps these (`make setup data train test lint`). Serving, Kubernetes and monitoring commands are in sections 8-10 and in the repository README.

4. Data acquisition and exploratory analysis

`scripts/download_data.py` fetches `processed.cleveland.data` from the UCI repository. The file has no header and marks missing values with `?`; `src/data.py` names the 14 columns, parses `?` as missing, and binarises the label (`num` 0 -> *no disease*, 1-4 -> *disease*). Only six values are missing (`ca`: 4, `thal`: 2); both columns are categorical, so they are filled with the column mode. The cleaned file has 303 rows and no missing values and is committed to the repository.

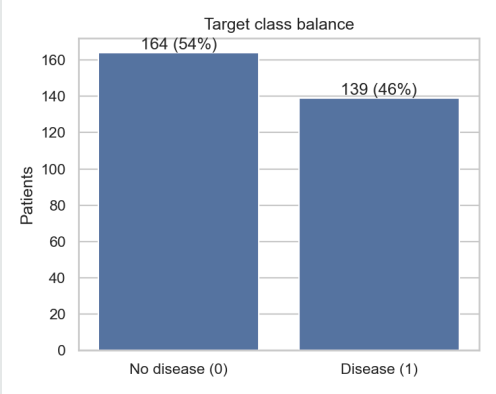


Figure 2 - Class balance: 164 no disease (54%), 139 disease (46%).

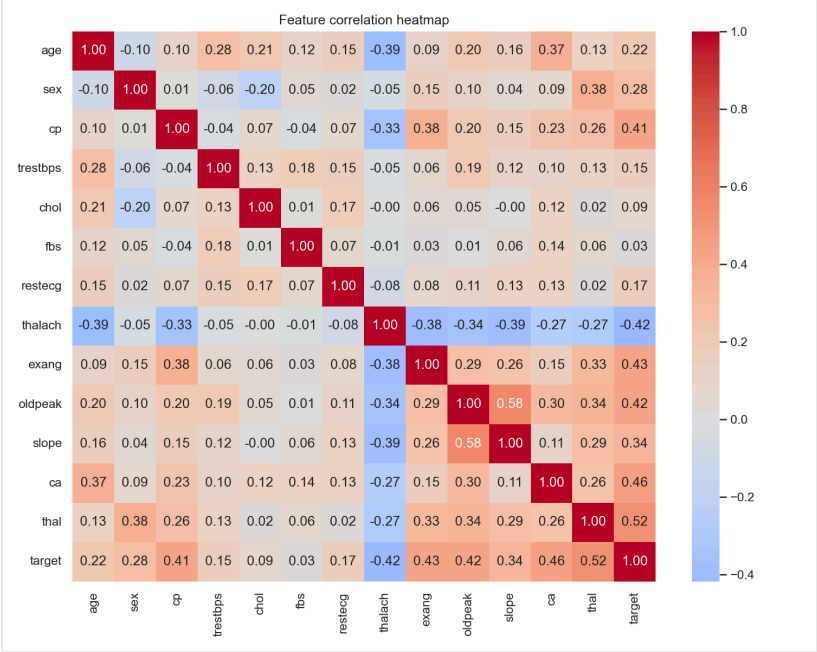


Figure 3 - Correlation heatmap including the target.

- **Balanced target** (54% / 46%), so accuracy is meaningful and no resampling is needed; recall and ROC-AUC are still reported because missing a sick patient is the costly error.
- **Strongest linear signals** with the target: `thal` (0.52), `ca` (0.46), `exang` (0.43), `oldpeak` (0.42), `thalach` (-0.42: a higher maximum heart rate means lower risk) and `cp` (0.41). `fbs` (0.03) and `chol` (0.09) are nearly uninformative on their own.
- `oldpeak` and `slope` are correlated (0.58): a mild collinearity that matters for linear-model coefficients, not for tree models.
- Numeric features sit on different scales, so scaling is required for Logistic Regression. Codes such as `cp`, `thal` and `slope` are nominal labels, not quantities, so they are one-hot encoded.

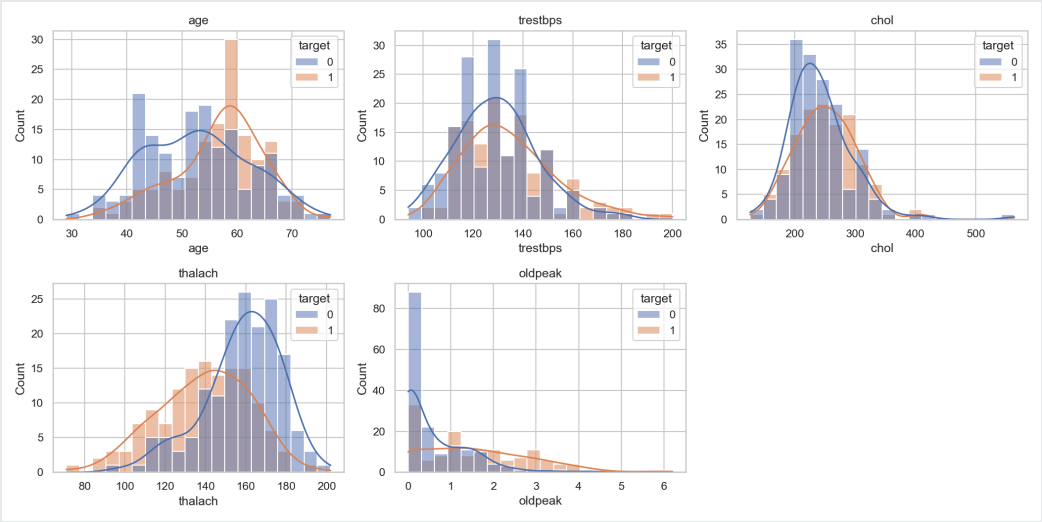


Figure 4 - Distributions of the numeric features, split by target (full notebook: notebooks/01_eda.ipynb).

5. Feature engineering and model development

Preprocessing lives in one `ColumnTransformer` (`src/features.py`) that is part of the model pipeline, so during cross-validation it is re-fitted on each training fold and never sees validation data. Numeric columns: median imputation then standardisation. Categorical columns: most-frequent imputation then one-hot encoding with `handle_unknown="ignore"`. The imputers are redundant on the cleaned CSV but make the served model robust to a missing or unseen value.

Protocol. A stratified 80/20 split (242 training, 61 test patients, seed 42). Each candidate is tuned with `GridSearchCV` (stratified 5-fold, scoring ROC-AUC) on the training split only; the tuned configuration is then re-scored with 5-fold cross-validation on accuracy, precision, recall, F1 and ROC-AUC. The model is **selected on cross-validated ROC-AUC**, and the test split is used once, for reporting.

Model	Grid searched	Best parameters
Logistic Regression	C in {0.01, 0.1, 1, 10}	C = 1
Random Forest	n_estimators {100, 300} x max_depth {None, 4, 8} x min_samples_leaf {1, 3}	100 trees, max_depth 4, min_samples_leaf 1
Gradient Boosting	n_estimators {100, 200} x learning_rate {0.05, 0.1} x max_depth {2, 3}	100 trees, learning_rate 0.05, max_depth 3

Model	CV accuracy	CV precision	CV recall	CV F1	CV ROC-AUC	Test acc.	Test recall	Test AUC
Logistic Regression	0.847	0.877	0.783	0.825	0.902	0.885	0.929	0.966
Random Forest	0.826	0.844	0.774	0.802	0.901	0.836	0.821	0.944
Gradient Boosting	0.793	0.771	0.783	0.775	0.875	0.885	0.929	0.951

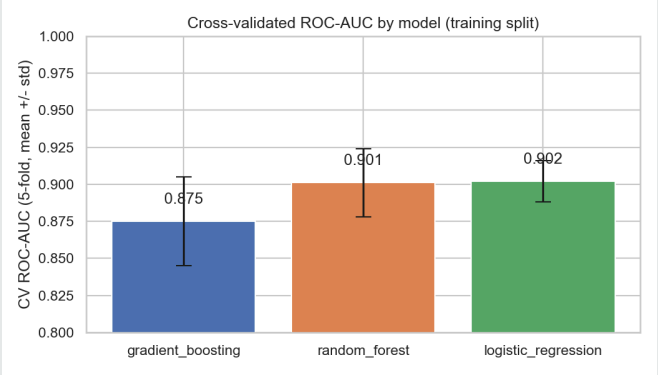


Figure 5 - Cross-validated ROC-AUC (mean +/- std).

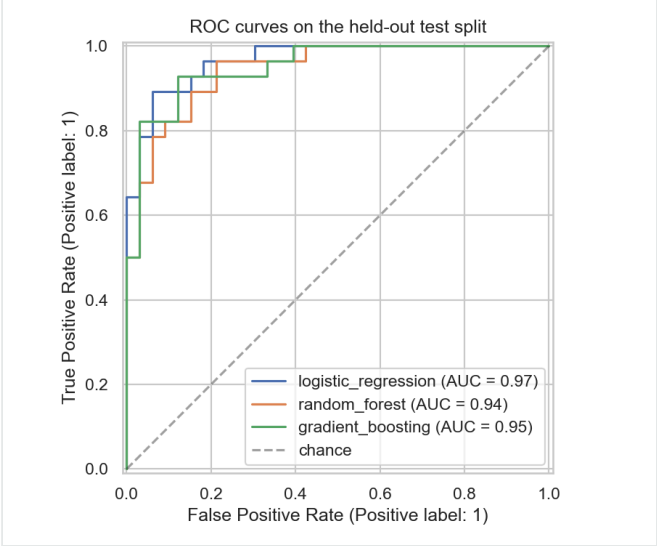


Figure 6 - ROC curves on the held-out test split.

Why Logistic Regression. It is marginally ahead on cross-validated AUC (0.902 vs 0.901), has the lowest fold-to-fold variance (0.014 vs 0.023), is the simplest and its coefficients can be inspected. The gap to Random Forest is far inside the noise, so I do not claim it is better. The held-out numbers are higher than the cross-validated ones for every model; with 61 patients one error moves accuracy by about 1.6 points, so the cross-validated figures (242 patients, five folds) are the more trustworthy estimate.

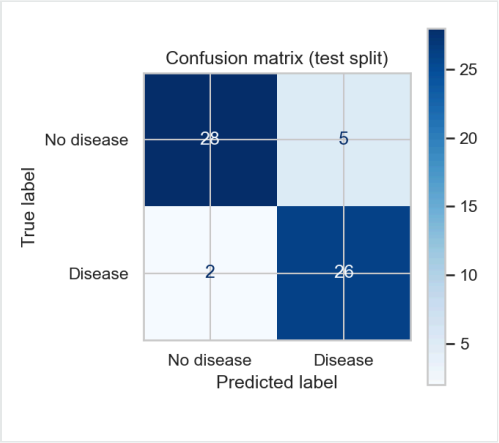


Figure 7 - Confusion matrix on the test split (selected model).

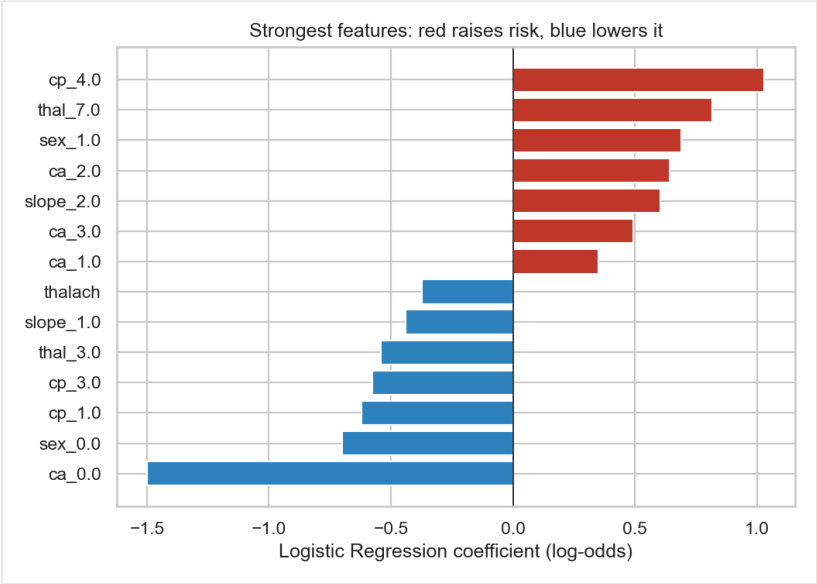


Figure 8 - Largest Logistic Regression coefficients.

The largest coefficients agree with the EDA: no coloured vessels (`ca_0`) is the strongest protective signal, while asymptomatic chest pain (`cp_4`) and a reversible thalassemia defect (`thal_7`) raise risk the most. Typical angina (`cp_1`) is counter-intuitively associated with *lower* predicted risk in this dataset - a pattern in the data, not clinical advice.

6. Experiment tracking with MLflow

`src/train.py` wraps every candidate in an MLflow run (SQLite backend) inside the experiment `heart-disease-classification`. Each run logs: the tuned hyper-parameters and the searched grid, 5-fold CV metrics (mean and standard deviation for five metrics), held-out test metrics, the confusion-matrix and ROC plots as artifacts, and the fitted pipeline as an MLflow model with an inferred signature and an input example. The selected run carries the tag `selected=true`. Because training is deterministic, `notebooks/02_training.ipynb` refits each model from its logged parameters and reproduces the logged test AUC exactly (0.9665, 0.9437, 0.9513).

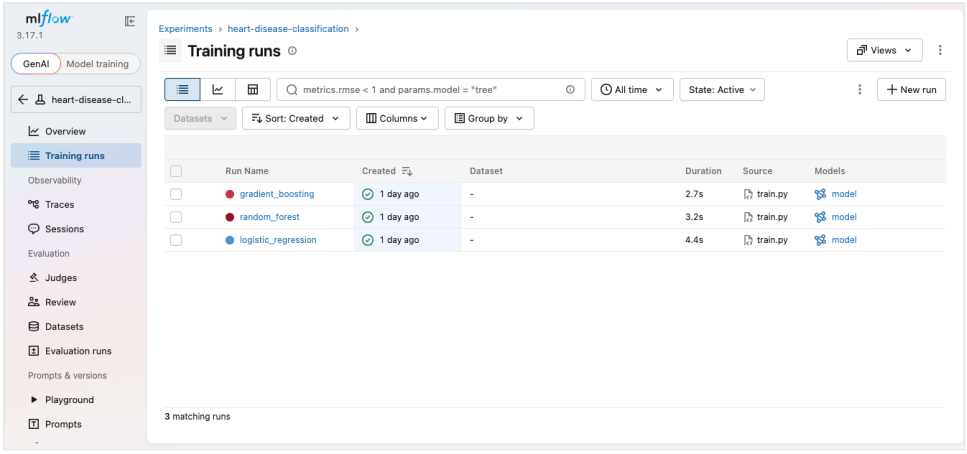


Figure 9 - MLflow experiment: one run per model, each with a logged model artifact.

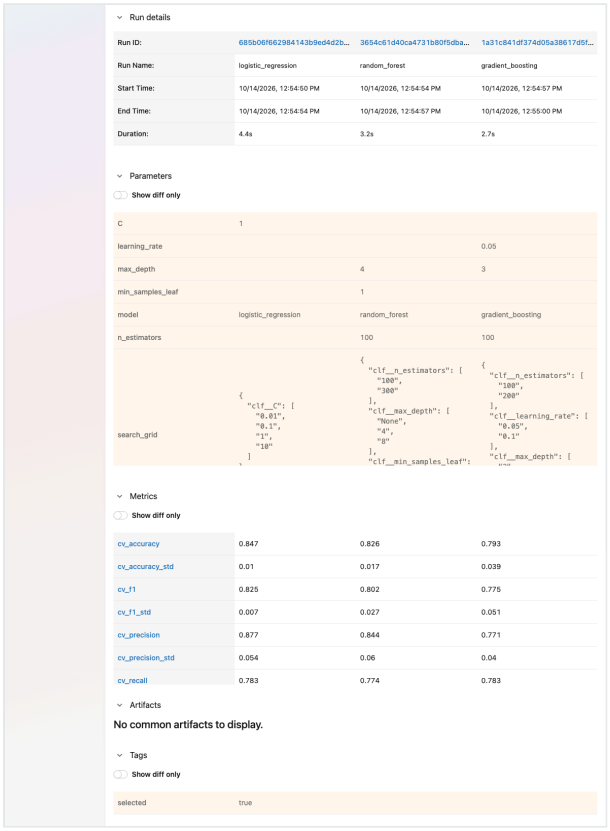


Figure 10 - Run comparison: parameters, metrics and the selected tag.

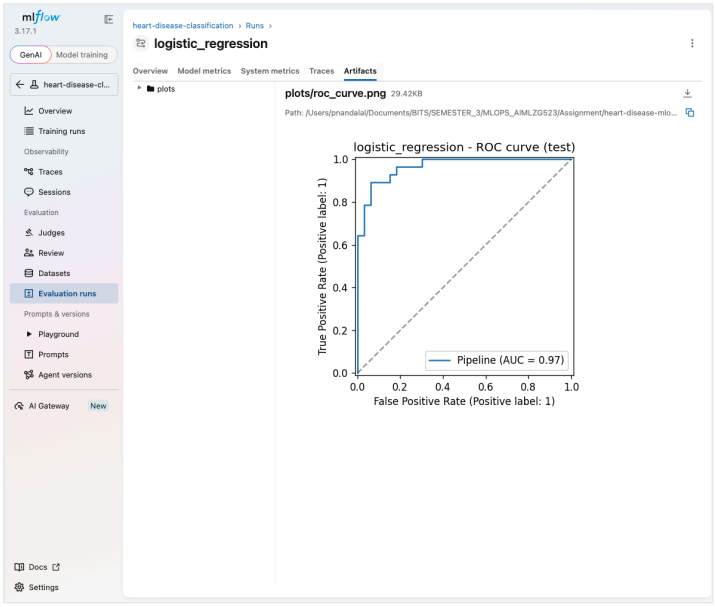


Figure 11 - A logged plot artifact (ROC curve).

7. Model packaging and reproducibility

- **One artifact for train and serve.** `models/model.joblib` is the *whole* pipeline (imputation, scaling, one-hot encoding, classifier). Inference takes raw patient fields; there is no separate preprocessing code that could drift from training. The same pipeline is also stored as an MLflow model.
- **Pinned environment.** `requirements.txt` pins every package; `requirements-serve.txt` holds only the runtime subset the container needs. Random seeds are fixed (42).
- **Verified from a clean setup.** The CI runner starts from an empty machine and installs only from `requirements.txt` before downloading the data, testing and training - so a green run is evidence that the scripts do not depend on anything local.

8. CI/CD pipeline and automated testing

The suite has **20 pytest tests**: data cleaning and parsing (5), the preprocessing transformers (3: finite output, standardised numeric columns, tolerance of unseen categories and missing values), the model (4: cross-validated

AUC above 0.8, output contract, determinism, saved model loads) and the API (8: health, prediction contract, four invalid-value cases, a missing field, metrics exposure).

`.github/workflows/ci.yml` runs on every push and on demand. Job **build-test-train**: install dependencies, `ruff` lint, download the data, `pytest`, train the models with MLflow, then upload the JUnit test report and the model, metrics and MLflow runs as workflow artifacts. Job **docker** (only if the first passes): build the image, start the container, and call `/health` and `/predict`. Steps are ordered cheapest-first and each depends on the previous one, so a failure stops the pipeline and marks the run red.

Run	Commit	Purpose	Result
#3	6bd7a67	Finished branch	**Passed** - lint, 20 tests, training, Docker build and smoke test
#4	1e90cec	Deliberate regression: target threshold changed to `> 1`	**Failed at Unit tests**; training and Docker jobs skipped
#5	1916dbc	Deliberate lint error (unused import)	**Failed at Lint**; tests, training, Docker skipped

Runs #4 and #5 were pushed on a throwaway branch (`ci-failure-demo`) to show that the pipeline really fails on code and test errors. The failing assertion was reproduced locally first (`assert 6 == 8`).

Screenshot to insert: `Figure 12` (see submission checklist)

Figure 12 - Run #3: all jobs and steps pass.

Screenshot to insert: `Figure 13` (see submission checklist)

Figure 13 - Workflow runs: green and deliberately red.

Screenshot to insert: `Figure 14` (see submission checklist)

Figure 14 - Run #4 stops at Unit tests.

Screenshot to insert: `Figure 15` (see submission checklist)

Figure 15 - Run #5 stops at Lint.

9. Containerisation and production deployment

Image. The Dockerfile starts from `python:3.10-slim`, installs only `requirements-serve.txt` (no MLflow or notebook tooling), copies just `src/`, `api/` and the model, runs as a non-root user and defines a `HEALTHCHECK`. The image is 473 MB.

API. `POST /predict` takes the 13 clinical fields as JSON and returns the prediction and the confidence (the probability of the predicted class). Inputs are validated (ranges and allowed codes); invalid or missing fields return HTTP 422 instead of a guess. `GET /health` is used by the probes and `GET /metrics` exposes Prometheus metrics.

```
$ curl localhost:8001/health
{"status":"ok"}

$ curl -X POST localhost:8001/predict -H 'Content-Type: application/json' -d '{"age":63,"sex":1,"cp":1,"tres
t...
{"prediction":0,"diagnosis":"no heart disease","confidence":0.8345,"probability_disease":0.1655}

$ curl -X POST localhost:8001/predict ... (high-risk patient)
{"prediction":1,"diagnosis":"heart disease","confidence":0.9856,"probability_disease":0.9856}

$ curl -X POST localhost:8001/predict -d {"age":-5}   # invalid input
{"detail":[{"type":"greater_than_equal","loc":["body","age"],"msg":"Input should be greater than or equal to
...
HTTP 422
```

Screenshot to insert: `Figure 16` (see submission checklist)

Figure 16 - Docker build and run with a sample request (terminal).

Kubernetes. The target is the local Docker Desktop cluster (context `docker-desktop`), chosen over a paid cloud so the deployment is free to reproduce. `k8s/` holds a Namespace, a Deployment (2 replicas, readiness and liveness probes on `/health`, CPU/memory requests and limits) and a `LoadBalancer` Service mapping port 8080 to the

container's 8000. The image uses `imagePullPolicy: Never` because it is built locally; on GKE/EKS/AKS it would be pushed to a registry and the policy changed.

```
kubectl --context docker-desktop apply -f k8s/
NAME                                 READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/heart-disease-api    2/2     2            2           43h

NAME                                     READY   STATUS    RESTARTS   AGE
pod/heart-disease-api-696dc4b5db-5gkxf   1/1     Running   0          43h
pod/heart-disease-api-696dc4b5db-ptbcx   1/1     Running   0          43h

NAME                        TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
service/heart-disease-api   LoadBalancer   10.101.240.117   localhost     8080:30731/TCP   43h
```

Screenshot to insert: `Figure 17` (see submission checklist)

Figure 17 - Pods and service on the cluster (terminal).

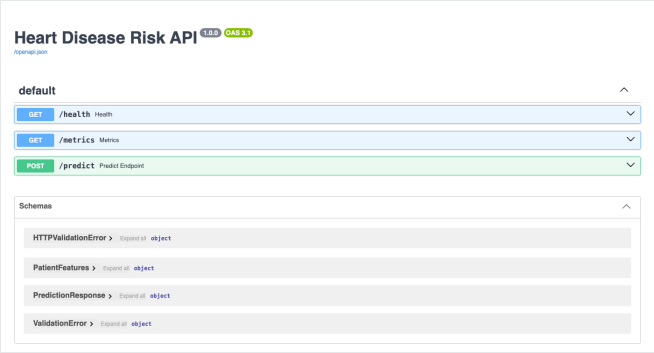


Figure 18 - API docs served through the LoadBalancer at localhost:8080.

10. Monitoring and logging

Every request is logged to stdout with method, path, status and latency, and every prediction with its input and result (visible with `kubectl logs`). The service exposes four Prometheus series: `api_requests_total{method,path,status}`, `api_request_latency_seconds` (histogram), `model_predictions_total{diagnosis}` and `model_prediction_errors_total`. `docker-compose.yml` starts the API, Prometheus (scraping every 5 s) and Grafana, which loads a provisioned dashboard: request rate by path and status, p50/p95 latency, predictions by outcome, and 4xx/5xx counts.

`scripts/generate_traffic.py` replays real dataset rows (with about 12% deliberately invalid requests) to exercise the dashboard. Under that local load the p95 `/predict` latency was about 24 ms. The 4xx counter rises with the invalid requests while 5xx stays at zero.

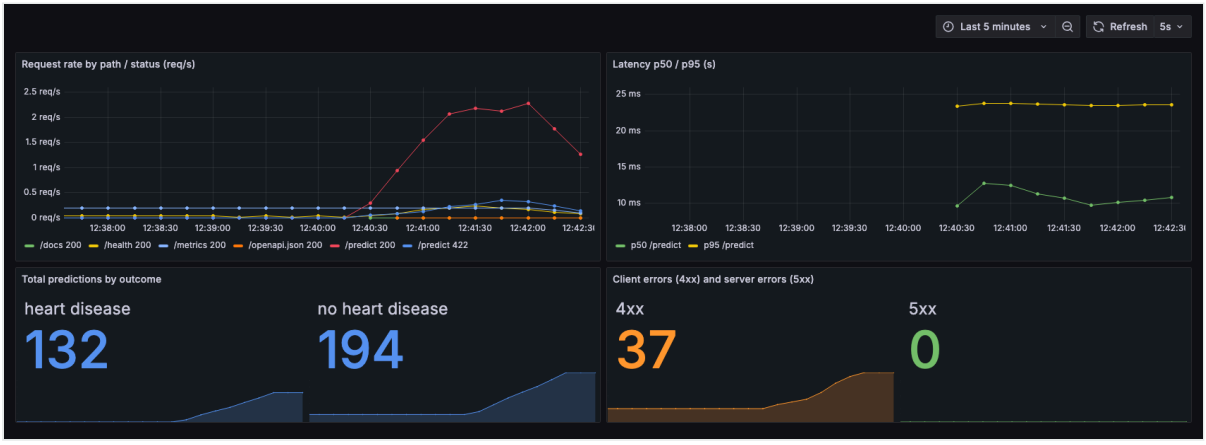


Figure 19 - Grafana dashboard during generated traffic.

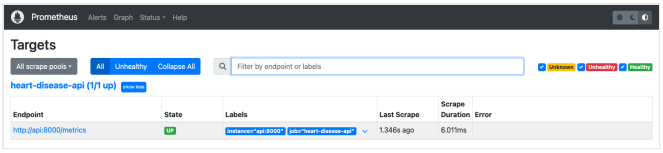


Figure 20 - Prometheus target `api:8000/metrics` is UP.

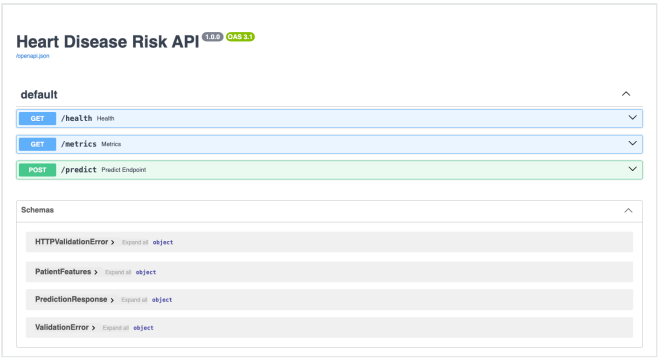


Figure 21 - Interactive API documentation (`/docs`).

11. Limitations and future work

- **Small data.** 303 patients give wide uncertainty; quote metrics with their standard deviations. This is a coursework model on a historical dataset and not a medical device.
- **Local deployment.** There is no public URL; the API is reachable at `localhost:8080` on the machine running the cluster. The model file is committed so images build reproducibly; a production system would load it from a model registry.
- **Not covered:** authentication and TLS, autoscaling, input-drift and performance monitoring, and automated retraining and continuous deployment (CI stops at a verified image).

12. Deliverables map

Required deliverable	Where
Code, Dockerfile, requirements	`src/`, `api/`, `Dockerfile`, `requirements.txt`, `requirements-serve.txt`
Cleaned dataset and download script	`data/processed/heart_clean.csv`, `scripts/download_data.py`
Notebooks (EDA, training, inference)	`notebooks/01_eda.ipynb`, `02_training.ipynb`, `03_inference.ipynb`
Unit tests	`tests/` (20 tests)
GitHub Actions workflow	`.github/workflows/ci.yml`
Deployment manifests	`k8s/`; monitoring in `monitoring/` and `docker-compose.yml`
Screenshots, report, video	`screenshots/`, `report/`, and the video link in the submission folder
Repository and API access	https://github.com/pnand5/heart-disease-mlops - API at http://localhost:8080 after `kubectl apply -f k8s/`